

PROJECT TITLE:	Bagify E-Commerce System
ARCHITECTURE:	Distributed Microservices (Spring Boot + React)
FRONTEND STACK:	React (Vite) + TypeScript + Tailwind CSS
BACKEND STACK:	Spring Boot, Eureka Service Registry, Spring Cloud Gateway
DATABASE:	PostgreSQL (Database-per-Service Pattern)
DOCUMENT TYPE:	Technical Review & Code Defence Study Guide

Introduction

This guide is prepared to help you defend your project in a code review or viva examination. It provides structured, detailed answers to technical questions about the architecture, security patterns, concurrency handlers, payment integration, and testing strategies implemented in **Bagify**.

Reviewing these questions will help you explain *why* you made certain design decisions (such as using a microservices pattern, pessimistic locking, or database denormalization) rather than just stating *what* the code does.

Core Technical Highlights of Bagify:

- **Service Discovery & Gateway:** Netflix Eureka server coordinates instances. Spring Cloud Gateway aggregates endpoints under port 8080 and handles CORS.
- **Database-per-Service:** High isolation where each service manages its own PostgreSQL schema, utilizing denormalization for order snapshots.
- **Pessimistic Concurrency Control:** Uses Spring Data JPA's `LockModeType.PESSIMISTIC_WRITE` to block double-checkout and race conditions during payment notifications.
- **Secure Payment Webhooks:** Validates HMAC/MD5 signatures received from PayHere (Sandbox) server-to-server redirects.
- **Downstream Fault Tolerance:** Leverages defensive try-catch boundaries to prevent statistics sync problems from rolling back payment completions.

1. Microservices Architecture & System Design

Q1: Why did you choose a microservices architecture for Bagify instead of a monolith? What are the key benefits and trade-offs in this design?

Answer: Monolithic applications bundle user profiles, product catalogs, and order management into a single deployable unit. In contrast, Bagify decomposes these domains into independent microservices (user, product, order, notification) to achieve:

1. **Fault Isolation:** If the order service goes down due to heavy load, buyers can still browse the catalog (product service) or register accounts (user service).
2. **Independent Scalability:** The product service (which receives heavy read traffic) can be scaled up independently from the heavier write-intensive order service.
3. **Domain Boundaries:** Each service handles its own isolated logic. Developers can work on the frontend or user profiles without risking order logic corruption.

Trade-offs: Microservices introduce distributed complexity: network latency between services, distributed transactions (ensuring stock is deducted when an order is paid), and operational overhead (managing multiple databases, ports, and discovery nodes).

Q2: What is the purpose of Eureka Server, and how do client microservices interact with it?

Answer: Eureka acts as a **Service Registry**. In a dynamic environment, services scale up and down, and their IP addresses or port numbers can change. Hardcoding physical URLs (like localhost:8082) is an anti-pattern. With Eureka:

1. At startup, each microservice registers its service name (e.g., spring.application.name=user) and coordinates with the Eureka server.
2. When the API Gateway or order service needs to make a call to the user service, it queries Eureka to resolve the application name to a live IP address and port.
3. It also enables client-side load balancing (via Spring Cloud LoadBalancer), routing requests among active instances.

Q3: Explain the role of the API Gateway (Spring Cloud Gateway) in Bagify. How does it handle routing and CORS?

Answer: The API Gateway serves as a single entry point (running on port 8080) for the frontend application. Instead of the React client connecting to multiple ports (8081 for user, 8082 for product, etc.), all requests go to the Gateway. The Gateway reads the request path and routes it using Eureka name-resolution:

- Paths matching /api/v1/auth/** are routed to lb://user.
- Paths matching /api/v1/products/** are routed to lb://product.
- Paths matching /api/v1/orders/** are routed to lb://order.

CORS: Cross-Origin Resource Sharing is configured exclusively at the Gateway. It permits incoming requests from the React client (http://localhost:5173) and sets allowed HTTP methods (GET, POST, PUT, DELETE) and headers globally. This eliminates the need to configure CORS inside individual downstream microservices.

Q4: Why is a 'Database-per-Service' pattern used? How do you query data across services, such as displaying the product name or seller details in an order history list?

Answer: Under the Database-per-Service pattern, each microservice owns its private database. The user service reads bagify_users, the product service reads bagify_products, and the order service reads bagify_orders. Sharing a database violates encapsulation, creates tight coupling, and leads to schema change bottlenecks.

Cross-Domain Queries: Direct database SQL joins are impossible. Instead, we use **Denormalization**. When an order is created, rather than storing just the productId and querying the product database at read-time, the order service stores a snapshot of the product details (name, image, and price at purchase time) inside the OrderItem entity. This not only preserves data independence but also ensures that order history remains historically accurate even if the seller later modifies a product's price or description.

2. Concurrency Control & Transaction Management

Q5: Explain how your order checkout process handles inventory management. What happens if two users try to buy the last remaining item at the same time?

Answer: The order checkout logic implements a two-phase check in `OrderService.placeOrder`:

1. **Validation:** It calls the product service to retrieve stock levels and verifies that the stock is greater than or equal to the requested quantity.
2. **Deduction:** If stock is sufficient, the order service sends an authenticated request to the product service's stock deduction endpoint to reduce inventory in a transaction.

To prevent race conditions where two users buy the last item simultaneously, the stock deduction query in the product service executes an atomic database operation, such as:

```
UPDATE products SET stock = stock - :quantity WHERE id = :id AND stock >= :quantity.
```

If the updated row count is 0, it indicates that another user's transaction committed first, causing the current transaction to fail and throw an `Insufficient stock` error.

Q6: How does the Order Service webhook handle concurrent notifications for the same order (e.g. duplicate signals from the payment gateway)?

Answer: When a payment is processed, the PayHere payment gateway makes a server-to-server POST request to the webhook endpoint (`/api/v1/orders/payment/notify`). Due to network retries or gateway behavior, duplicate notifications may arrive at the same millisecond. To guarantee **Idempotency** and prevent race conditions, the system uses **Pessimistic Write Locking**.

The repository method `findByIdForUpdate` is marked with `@Lock(LockModeType.PESSIMISTIC_WRITE)`. When the first thread calls this method, JPA executes a `SELECT ... FOR UPDATE` query, locking the database row. The second thread is blocked and must wait.

```
// In OrderRepository.java
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT o FROM Order o WHERE o.id = :id")
Optional<Order> findByIdForUpdate(@Param("id") Long id);
```

Once the first thread releases the lock by committing the transaction, the second thread acquires it. The second thread evaluates the local payment status flag immediately:

```
boolean isAlreadyPaid = "PAID".equals(order.getPaymentStatus());
```

If it is already PAID, the service logs a duplicate event warning and immediately returns without executing any state transitions, database saves, or downstream microservice calls.

Q7: In `OrderService.processPaymentNotification`, the call to update user stats is wrapped in a try-catch. What is the design rationale behind this?

Answer: The payment webhook handles critical financial state transitions (marking an order as PAID). After marking the order as paid, the system tries to update the seller's statistics in the user microservice via an HTTP call. If the user microservice is temporarily down or experiences a network timeout, propagating the exception would roll back the entire transaction. This would revert the order to UNPAID in our database, even though the buyer has already paid and the gateway has confirmed receipt of funds. We wrap the call in a try-catch block to maintain system reliability, log the failure, and allow the core transaction (saving the paid order) to complete successfully.

```
// Update seller stats in user microservice
try {
    Integer sellerId = Integer.parseInt(item.getSellerId());
    double itemRevenue = item.getPriceAtPurchase() * item.getQuantity();
    userClient.updateSellerStats(sellerId, itemRevenue, item.getQuantity());
} catch (Exception e) {
    log.error("Could not parse seller ID or call UserClient: {}", e.getMessage());
    // Exception caught: order remains marked as PAID!
}
```

3. Security & Token Propagation

Q8: How is JWT-based authentication implemented and validated across your microservices? How is the user profile extracted?

Answer: The architecture uses stateless JWT (JSON Web Token) authentication:

1. **Generation:** The user service generates a JWT signed with a secret key upon a successful login. The token contains claims: `userId`, `email`, and `roles`.
2. **Validation:** Downstream services (e.g., `order`, `product`) contain a `JwtFilter` that intercepts incoming HTTP requests, extracts the `Authorization: Bearer <token>` header, decodes it using the shared secret, and validates its expiry and signature.
3. **Context Extraction:** Once validated, the filter parses the roles (e.g., `ROLE_BUYER`) and user ID, and populates the Spring Security `SecurityContext`. Controllers access the user ID via `authentication.getDetails()`.

Q9: What is 'Token Propagation' and how is it used during the checkout process?

Answer: When a buyer calls the order service to check out, the request is authenticated with their JWT. However, to validate stock and deduct inventory, the order service must make a synchronous REST call to the product service. The product service endpoints are secured and require authentication. **Token Propagation** is the pattern where the order service extracts the user's incoming JWT and forwards it in the headers of its outgoing HTTP client (`WebClient` or `RestTemplate`) to the product service. This maintains security context across microservice boundaries.

```
// In OrderController.java - Forwarding the JWT
String bearerToken = httpRequest.getHeader("Authorization");
OrderResponse response = orderService.placeOrder(buyerId, request, bearerToken);

// In ProductClient.java - Passing the Bearer Token
public void deductStock(Long productId, int quantity, String bearerToken) {
    webClient.put()
        .uri("/api/v1/products/" + productId + "/deduct-stock?quantity=" + quantity)
        .header("Authorization", bearerToken) // Token propagated here!
        .retrieve()
        .toBodilessEntity().block();
}
```

Q10: How do you restrict specific endpoints to Sellers or Admins only?

Answer: We configure method-level security in our Spring Boot configuration using `@EnableMethodSecurity` and apply the `@PreAuthorize` annotation. For example, updating the status of an order item is restricted to sellers, whereas overriding an item's status to `DELIVERED` is restricted to admins.

```
@PutMapping("/{orderId}/items/{itemId}/status")
@PreAuthorize("hasRole('SELLER')")
public ResponseEntity<OrderResponse> updateItemStatus(...) { ... }

@PutMapping("/{orderId}/items/{itemId}/status/admin")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<OrderResponse> updateItemStatusAdmin(...) { ... }
```

4. Payment Gateway Integration (PayHere)

Q11: How do you prevent a user from tampering with order details (like changing the currency or reducing the amount) before redirecting to the PayHere payment page?

Answer: We generate an MD5 cryptographic signature hash on the backend, which is passed to the frontend and submitted directly to PayHere. The hash is calculated as:

MD5(Merchant ID + Order ID + Formatted Amount + Currency + MD5(Merchant Secret)).

When the user is redirected, PayHere's servers recalculate the hash using the parameters sent and the merchant secret known only to PayHere and our backend. If the user tampered with the price in the browser, the hashes will not match, and PayHere will reject the transaction immediately. The merchant secret is stored in application properties and is never exposed to the frontend.

Q12: Explain the validation steps performed when receiving the server-to-server webhook callback from PayHere.

Answer: The callback in `OrderService.processPaymentNotification` implements strict verification steps:

1. **Parameter Check:** It verifies that all required parameters (merchant ID, order ID, amount, currency, status code, and the signature hash md5sig) are present.
2. **Signature Recalculation:** It generates a signature hash of the incoming parameters using the formula: MD5(Merchant ID + Order ID + PayHere Amount + PayHere Currency + Status Code + MD5(Merchant Secret)).
3. **Case-Insensitive Match:** It compares our calculated hash with the md5sig sent by PayHere, ignoring case (to handle variance in hexadecimal formatting between systems). If they don't match, it throws a 400 Bad Request error.
4. **Order Fetch and Lock:** It fetches the corresponding order using a pessimistic lock (preventing concurrent updates).
5. **Payment Evaluation:** It checks if the order has already been marked as PAID to prevent duplicate processing.

Q13: How is the PayHere signature generator implemented in your utility classes?

Answer: It uses Java's `MessageDigest` library to calculate the MD5 hash and formats the resulting bytes into a hexadecimal string.

```
public static String generateMd5(String source) {
    try {
        MessageDigest md = MessageDigest.getInstance("MD5");
        byte[] messageDigest = md.digest(source.getBytes());
        StringBuilder sb = new StringBuilder();
        for (byte b : messageDigest) {
            sb.append(String.format("%02x", b));
        }
        return sb.toString().toUpperCase();
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException(e);
    }
}
```

5. Frontend Architecture & State Management

Q14: How is state management structured in the React frontend? What is the role of Context Providers?

Answer: The React application uses the Context API for lightweight, global state management without the boilerplate of Redux. Context Providers wrap the component tree in App.tsx:

- **AuthProvider:** Manages the active user's session, JWT, role, and details, persisting them to localStorage.
- **ProductProvider:** Exposes the catalog, search queries, category filters, and product CRUD methods.
- **CartProvider:** Tracks items added to the shopping cart, calculates subtotals, and facilitates checkout.
- **WishlistProvider:** Keeps track of items the buyer has bookmarked.

Q15: How are routes secured in React? How do you prevent a buyer from manually accessing the admin dashboard via URL manipulation?

Answer: The React application uses the Context API for lightweight, global state management without the boilerplate of Redux. Context Providers wrap the component tree in App.tsx:

- **AuthProvider:** Manages the active user's session, JWT, role, and details, persisting them to localStorage.
- **ProductProvider:** Exposes the catalog, search queries, category filters, and product CRUD methods.
- **CartProvider:** Tracks items added to the shopping cart, calculates subtotals, and facilitates checkout.
- **WishlistProvider:** Keeps track of items the buyer has bookmarked.

Q16: Why did you choose Vite over Create React App (CRA)?

Answer: Vite is a modern frontend build tool. It uses native ES modules in development, enabling extremely fast server starts and instant Hot Module Replacement (HMR) regardless of project size. Production builds are powered by Rollup, which produces cleaner, smaller bundles than Create React App's Webpack configuration. Additionally, Vite is highly configurable and includes built-in support for TypeScript out of the box.

6. Testing Strategy

Q17: What was your strategy for testing the payment webhook logic? What edge cases did you cover?

Answer: We wrote Mockito unit tests in OrderServiceWebhookTest.java to isolate the service logic from the database and external REST clients. The tests validated several scenarios:

1. **Successful Webhook:** Mocked a valid signature and status code 2. Verified that the order transitioned to PROCESSING and the payment status to PAID.
2. **Tamper Detection:** Simulated a request where the payment amount was altered, and verified that hash validation failed.
3. **Idempotency (Duplicate Request):** Passed an order that was already marked as PAID, and verified that the service bypassed saving the order or invoking downstream APIs.
4. **Downstream Failure:** Configured the user client to throw a network exception during the update, and verified that the order was still successfully saved as paid.


```
// Testing duplicate webhooks
@Test
void testProcessPaymentNotification_DuplicatePaidWebhook_SuccessStatus() {
    // Arrange: Order is already marked as PAID
    Order order = new Order();
    order.setPaymentStatus("PAID");
    when(orderRepository.findByIdForUpdate(45L)).thenReturn(Optional.of(order));

    // Act: Process the webhook
    orderService.processPaymentNotification(params);

    // Assert: Verify order save and user client calls were bypassed
    verify(orderRepository, never()).save(any());
    verifyNoInteractions(userClient);
}
```